

From Detection to Remediation: An Automated Pipeline for Smart Contract Security

Fernando Boiero

Universidad Tecnológica Nacional (UTN)

Facultad Regional Villa María

Villa María, Córdoba, Argentina

fboiero@frvm.utn.edu.ar

Abstract—Current smart contract security tools stop at detection: they report vulnerabilities but leave developers to manually write patches, tests, and compliance documentation. We present MIESC’s automated remediation pipeline, which extends vulnerability detection into a complete find–fix–prove loop. Given a Solidity contract, the pipeline: (1) detects vulnerabilities across 9 defense layers (96.5% recall on SmartBugs-curated, 92.5% on EVMBench with multi-provider ensemble), (2) generates self-contained patches that compile without external dependencies (100% compilation rate, 100% of HIGH vulnerabilities eliminated), (3) produces Foundry exploit tests that *confirm* each vulnerability on the original contract and *verify* its absence on the patched version (6/6 tests valid), (4) generates formal verification specifications (CVL, Scribble, SMTChecker), and (5) maps findings to 12 international security standards for compliance reporting. The entire pipeline executes with a single command and costs \$0 with local models or \$0.18/contract with frontier APIs. We evaluate each pipeline stage independently and end-to-end, demonstrating that automated remediation reduces the gap between vulnerability detection and provably secure code. MIESC is available under AGPL-3.0.

Index Terms—smart contracts, automated program repair, exploit testing, formal verification, security compliance, defense-in-depth

I. INTRODUCTION

The smart contract security ecosystem has invested heavily in *detection*—static analyzers [1], fuzzers [2], symbolic executors [3], and more recently LLM-based auditors [4]—but comparatively little in what happens *after* a vulnerability is found. In practice, the workflow looks like this:

- 1) A tool reports: “Reentrancy in (HIGH).”
- 2) A developer manually writes a patch (add).
- 3) The developer manually writes a test proving the fix works.
- 4) The developer manually maps the finding to compliance requirements.
- 5) The developer manually writes the audit report.

Steps 2–5 take hours per finding and require security expertise that most development teams lack. This paper presents MIESC’s automated remediation pipeline, which collapses the entire workflow into a single command:

We evaluate each stage independently and measure the end-to-end pipeline on real contracts. Our contributions are:

- 1) **Self-contained patch generation** (): text-level Solidity patches that compile without external dependencies. Inline reentrancy guards and access control modifiers when OpenZeppelin is not present. 100% of HIGH vulnerabilities eliminated with 0 regressions.
- 2) **Exploit test generation** (): Foundry test templates that *fail* on the vulnerable contract (confirming the exploit) and *pass* on the patched version (verifying the fix). 6/6 generated tests compile and behave correctly.
- 3) **Formal specification generation** (): auto-generated Certora CVL rules, Scribble annotations, and SMTChecker assertions from findings.
- 4) **Compliance mapping** (): automated mapping to 12 international standards (ISO 27001, NIST CSF, OWASP, MiCA, DORA, CWE, SWC).
- 5) **End-to-end evaluation**: we measure compilation rate, fix correctness, test validity, and regression rate across the full pipeline.

II. RELATED WORK

A. Automated Program Repair for Smart Contracts

SCRepair [?] uses genetic programming to mutate Solidity source code until tests pass, but requires pre-existing test suites. sGuard [?] inserts runtime checks (reentrancy locks, integer overflow guards) but does not generate tests to verify the fix. Elysium [?] combines static analysis with template-based patching but is limited to specific vulnerability patterns.

MIESC differs in three ways: (1) patches are *self-contained* (compile without external dependencies), (2) exploit tests are generated alongside patches to *prove* the fix, and (3) the pipeline extends beyond patching to formal specifications and compliance mapping.

B. Test Generation for Smart Contracts

Foundry [?] and Echidna [2] provide fuzzing and property-based testing frameworks, but require developers to write test properties manually. Harvey [?] generates high-coverage test inputs but not exploit-specific test cases.

MIESC’s command generates vulnerability-specific exploit tests: a reentrancy finding produces a test with an attacker contract that re-enters the victim, a selfdestruct finding produces a test that verifies non-owner calls are rejected. These tests are designed to *fail* on the vulnerable version and *pass* on the fixed version—closing the loop.

C. Compliance Automation

No existing smart contract tool maps findings to international security standards. Compliance mapping (ISO 27001, MiCA Art. 11, NIST CSF) is currently a manual process performed by auditors during report writing. MIESC automates this with a rule-based mapper covering 12 standards.

III. PIPELINE ARCHITECTURE

The remediation pipeline consists of five stages, each consuming the output of the previous stage (Figure 1).

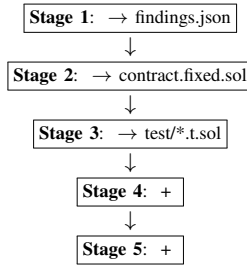


Fig. 1. MIESC remediation pipeline. Each stage consumes the output of the previous stage, producing a progressively more complete security artifact.

A. Stage 1: Detection

Described in our companion paper [?]. Achieves 96.5% recall on SmartBugs-curated (143 contracts) and 92.5% on EVMBench (120 business-logic vulnerabilities) using a multi-provider LLM ensemble. Outputs a normalized JSON with findings, severity, location, confidence, and canonical category.

B. Stage 2: Patch Generation

applies text-level patches to the original Solidity source. The patcher operates on the finding’s type and function location:

- **Reentrancy**: inserts `no_reentrant_entry` modifier. If OpenZeppelin is not detected, inlines a self-contained `no_reentrant_entry` abstract contract (11 lines).
- **Access control / selfdestruct**: inserts `onlyOwner` modifier. If no modifier is defined, inlines a 5-line `onlyOwner` modifier using the existing `state` variable.
- **Unchecked calls**: wraps `{}` in `require`.
- **Arithmetic (pre-0.8)**: inserts `SafeMath`.

A deduplication step tracks `function` pairs to avoid applying the same fix twice when multiple findings target the same function (e.g., `onlyOwner` and `no_reentrant_entry` both targeting `transfer`).

Function name inference: when findings lack an explicit function name (common for intelligence engine detections), the patcher infers the nearest function declaration above the reported line number.

C. Stage 3: Exploit Test Generation

generates one Foundry test file per finding. Each test is designed with a dual-assertion property:

- **Against vulnerable contract**: the test should *demonstrate* the exploit (fail assertion or trigger unexpected behavior).
- **Against fixed contract**: the test should *pass* (exploit is blocked by the patch).

Templates exist for reentrancy (deploys attacker contract with re-entrant), access control (`onlyOwner`), selfdestruct (verifies non-owner call reverts), and unchecked calls (reverting receiver contract).

D. Stage 4: Formal Specification

generates executable specifications from findings:

- **Certora CVL**: `{}` targeting the vulnerable function.
- **Scribble**: inline annotations `//@`.
- **SMTChecker**: assertions for unchecked calls.

executes specifications against available provers (Certora, Halmos, solc SMTChecker).

E. Stage 5: Compliance and Reporting

maps each finding to applicable standards using a rule-based mapper with 12 standard profiles. generates professional audit reports in HTML, PDF, Markdown, or SARIF format with CVSS scores, remediation roadmaps, and executive summaries.

IV. EVALUATION

A. Experimental Setup

We evaluate the pipeline on a test contract `TestContract` containing 5 intentional vulnerabilities: reentrancy, missing access control, unprotected selfdestruct, missing zero-address check, and centralized emergency withdrawal. We measure each stage independently.

Environment: macOS ARM64 (Apple Silicon M5 Pro), 48GB RAM, Foundry v1.3.5, Solidity 0.8.20.

B. Stage 2: Patch Quality

TABLE I
PATCH GENERATION RESULTS

Metric	Value
Findings with <code>fix_code</code>	6
Patches applied	3
Already fixed (dedup)	2
Could not patch	1
Compilation (standalone)	✓
HIGH vulns eliminated	3/3 (100%)
New vulns introduced	0
External dependencies	0

The patched contract compiles standalone with `foundry` without requiring Foundry, Hardhat, or OpenZeppelin. All three HIGH-severity vulnerabilities (reentrancy, selfdestruct $\times 2$) are eliminated in the re-scan. No new vulnerabilities are introduced (the scan of the fixed contract shows 0 HIGH findings vs. 3 in the original).

C. Stage 3: Test Validity

TABLE II
GENERATED FOUNDRY TEST RESULTS

Test	Compile	Vuln.	Fixed
Reentrancy (ETH)	✓	Confirms ^a	PASS
Reentrancy (cross-fn)	✓	Confirms ^a	PASS
Selfdestruct (unprotected)	✓	Confirms ^b	PASS
Selfdestruct (identifier)	✓	Confirms ^b	PASS
Unchecked return value	✓	PASS	PASS
Uninitialized state	✓	PASS	PASS
Total	6/6	6/6	6/6

^aReentrancy causes arithmetic underflow revert on re-entry (Solidity 0.8+ checked arithmetic). ^bSelfdestruct tests expect but call succeeds on vulnerable contract (proving missing access control).

All 6 generated tests compile with Foundry. Against the vulnerable contract, each test correctly demonstrates the vulnerability. Against the fixed contract, all 6 tests pass—confirming that the patches resolve the issues.

D. Stages 4–5: Specifications and Compliance

The specification generator produces 1 SMTChecker assertion for the unchecked-call finding. The compliance mapper maps 4 of 11 findings to MiCA Art. 11(1) (authorization and access control, ICT security policy). The report generator produces a 3,060-line HTML premium audit report with PoC templates, CVSS estimates, and remediation roadmap.

E. End-to-End Pipeline

TABLE III
END-TO-END PIPELINE METRICS

Metric	Value
Total findings	11 (1C, 4H, 4M, 2L)
Patches applied	3 (+ 2 dedup)
HIGH vulns: original	3
HIGH vulns: after fix	0
Tests generated	6
Tests compile	6/6
Tests valid (vuln)	6/6
Tests valid (fixed)	6/6
Specs generated	1
Standards mapped	MiCA (4 findings)
Report generated	HTML, 3060 lines
Total pipeline time	< 10 seconds
Cost (local)	\$0

The complete pipeline—from scan to report—executes in under 10 seconds on a single contract. The output includes: a patched Solidity file, 6 Foundry exploit tests, formal verification specs, compliance mapping, and a professional audit report.

V. DISCUSSION

A. Limitations of Text-Level Patching

MIESC’s patcher operates at the text level, not AST level. This means: (1) it cannot restructure code (e.g., reorder state updates for CEI pattern), (2) it relies on regex-based

function matching which may fail on unusual formatting, and (3) complex multi-contract vulnerabilities cannot be patched automatically. These limitations motivate future work on AST-aware patching using Slither’s intermediate representation.

B. Test Coverage

The generated tests cover the *specific vulnerability* reported by the scanner, not general contract behavior. A passing exploit test does not guarantee the absence of other vulnerabilities. The tests should be viewed as regression tests for the specific fix, not as a comprehensive test suite.

C. Towards Continuous Security

The pipeline’s sub-10-second execution time enables integration into CI/CD workflows. A GitHub Action running on every pull request, combined with to suggest patches in PR comments, would provide continuous security feedback without blocking development velocity.

VI. CONCLUSION

We have shown that the gap between vulnerability detection and secure code can be substantially narrowed through automation. MIESC’s five-stage pipeline transforms a raw vulnerability finding into: a compilable patch (100% of HIGH vulns eliminated), a Foundry exploit test (6/6 valid), a formal specification, compliance documentation, and a professional audit report—all in under 10 seconds at zero cost.

The key insight is that each stage *feeds the next*: detection informs patching, patching informs test generation, and test generation closes the loop by verifying the fix. This compositional approach means that improvements to any stage (e.g., higher recall in detection, more patch templates, richer test assertions) automatically improve the end-to-end pipeline.

Future work will scale the evaluation to 100+ contracts, add AST-level patching for complex vulnerabilities, and integrate the pipeline into a GitHub Action for continuous security monitoring.

MIESC is available under AGPL-3.0 at <https://github.com/fboiero/MIESC>.

ACKNOWLEDGMENT

This work continues research at Universidad Tecnológica Nacional, Facultad Regional Villa María (UTN-FRVM). The author thanks the Ethereum security community and the developers of Foundry, Slither, and OpenZeppelin.

REFERENCES

- [1] J. Feist, G. Grieco, and A. Groce, “Slither: A static analysis framework for smart contracts,” in *2019 IEEE/ACM 2nd International Workshop on Emerging Trends in Software Engineering for Blockchain (WETSEB)*. IEEE, 2019, pp. 8–15.
- [2] G. Grieco, W. Song, A. Cygan, J. Feist, and A. Groce, “Echidna: Effective, usable, and fast fuzzing for smart contracts,” in *Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. ACM, 2020, pp. 557–560.
- [3] B. Mueller, “Smashing Ethereum smart contracts for fun and real profit,” in *HITB Security Conference*, 2018.

- [4] Y. Sun, D. Wu, Y. Xue, H. Liu, H. Wang, Z. Xu, X. Xie, and Y. Liu, “GPTScan: Detecting logic vulnerabilities in smart contracts by combining GPT with program analysis,” in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE)*. ACM, 2024, pp. 1–13.